

SoloPlate Reflective Report

Domain understanding

The problem I chose is meal planning for one person. At first, I thought the main feature should simply be a list of food in the fridge. After thinking about my own experience, I found that a list alone does not solve the difficult part. I have bought corn and sweet potatoes several times, but I was not sure how to use them in a simple meal. Preparing them also felt troublesome, so I kept leaving them for later. Some of this food eventually spoiled and I threw it away. This changed my design because the app now connects each fridge item to a practical meal suggestion instead of only storing information.

I also decided that SoloPlate should not tell the user whether food is safe. The app cannot see, smell or check the real ingredient. It only knows the date and quantity entered by the user. The screen therefore says that the date is used for sorting, and the user still needs to check the real food. This keeps the system boundary clear and avoids giving confidence that the app cannot support.

Architecture decisions

I used SwiftUI with MVVM and added a Use Case layer between the ViewModel and repositories. I chose three Use Cases because they match the main actions in the real workflow.

RegisterFridgeItemUseCase checks a new food record. RecommendTonightMealsUseCase finds suitable recipes. RecordPreparedMealUseCase updates the fridge after the user has cooked.

These Use Cases protect rules that matter for the stakeholder. A food name cannot be blank and its quantity must be above zero because an incomplete record cannot match a recipe properly. A recommended recipe must serve one person, take no more than 15 minutes, and have enough recorded ingredients. Recipes using food with an earlier date appear first. These rules support the original reason for the app, which is making a quick decision and using food before it is forgotten.

The repositories keep storage details outside the Use Cases. For this MVP, the fridge is held in memory and recipes come from a local JSON file. This is enough to demonstrate the business flow without adding accounts, a server or an external recipe API. It also makes the Use Cases easier to test with small sets of sample data.

Human-system design

One important mistake can happen when the digital fridge does not match the real fridge. The user may enter the wrong quantity or use an ingredient without updating the app. A recipe could then appear possible even though there is not enough food when cooking starts.

I handled this problem at the confirmation step. Looking at a recipe does not change the inventory. The user must select Meal Prepared after cooking. The Use Case then checks every required ingredient again. It calculates all changes before saving anything. If one ingredient is missing or has an insufficient quantity, the app explains which food caused the problem and tells the user to update My Fridge or choose another meal. The original quantities remain unchanged. This is important because partially reducing some ingredients would make the recorded fridge even less accurate.

The error messages use food and meal language instead of technical terms. For example, the app says that there are not enough eggs recorded rather than showing a generic data error. The user can understand what happened and what to do next.

What I would do next

With another two weeks, I would add simple local persistence for the fridge, probably using SwiftData. At the moment, the in-memory repository resets when the app closes. This is acceptable for demonstrating the MVP flow, but a real user needs the fridge list to remain available the next day.

I would keep the repository protocol and replace the current in-memory implementation with a SwiftData repository. The Use Cases and most tests could stay the same because they depend on the repository behaviour instead of the storage technology. I would also add tests for saving and loading data before changing the UI. This would improve the usefulness of SoloPlate without changing its small purpose or adding unnecessary online services.